

Assignment 4 – Frequent itemset mining

Description

We are going to implement frequent itemset mining over both MySQL and MongoDB.

Your tasks

1.- Provide queries that prepare the data (transactions and items). These transactions are represented as follows in MySQL:

T(tid, iid, PK(tid, iid))

And as follows in MongoDB:

T: {_id: tid, items: [iid_1, iid_2, ...]}

There will be two main datasets for transactions as follows:

1.1.- For a given business category, transactions are businesses and items are (unique) users who reviewed those businesses.

1.2.- For a given range of “yelping_since” dates, transactions are users and items are (unique) businesses they are reviewed.

In MongoDB, it is highly encouraged to create indexes for the items as follows: db.T.createIndex({items: 1}). Check the grading software and query templates for more info.

2.- Provide queries that compute L1 from the transactions described in Question 1. In MySQL, L1 is as follows:

L1(iid_1, count)

And as follows in MongoDB:

L1: {item_1: iid, count: z}

Only tuples and documents such that their count is greater or equal than the minimum support provided as input will remain. Check the grading software and query templates for more info.

3.- Provide queries that implement the join step: it computes Ck based on Lk-1. In MySQL, Ck is as follows:

Ck(iid_1, iid_2, ..., iid_k)

And as follows in MongoDB:

Ck: {iid_1: value, iid_2: value, ..., iid_k: value}

Item ids must be sorted, i.e., the value in iid_(i-1) must be strictly less than the value in iid_i. Check the grading software and query templates for more info.

4.- Provide queries that implement the prune step: it prunes Ck based on Lk-1. Check the grading software and query templates for more info.

5.- Provide queries that generates Lk using Ck and transactions. In MySQL, Lk is as follows:

`Lk(iid_1, iid_2, ..., iid_k, count)`

And as follows in MongoDB:

`Lk: {iid_1: value, iid_2: value, ..., iid_k: value, count: c}`

Only tuples and documents such that their count is greater or equal than the minimum support provided as input will remain. Check the grading software and query templates for more info.

6.- Provide queries that compute the association rules derived from a given Lk based on Lk-1 and L1 provided also as input. For each rule, you must compute support, confidence and lift. Check the grading software and software templates for more info.

7.- Provide queries to create a vertical L1 In MySQL, vertical L1 is as follows:

`L1Vertical(iid_1, count, transactions, eclass)`

And as follows in MongoDB:

`L1Vertical: {iid_1: value, count: z, transactions: [tid_1, tid_2, ...], eclass: c}`

where 'transactions' contains the transaction ids in which the current item appears. We will use JSON arrays in both MySQL and MongoDB. The 'eclass' attribute/field indicates the equivalence class, which is the same number for all the items in L1. Check the grading software and query templates for more info.

8.- Provide queries to implement the join step from vertical Lk-1 to vertical Lk. In MySQL, vertical Lk is as follows:

`Lk_Vertical(iid_1, iid_2, ..., iid_k, count, transactions, eclass)`

And as follows in MongoDB:

`Lk_Vertical: {iid_1: value, iid_2: value, ... iid_k: value, count: z, transactions: [tid_1, tid_2, ...], eclass: c}`

Check the grading software and query templates for more info.

Submission instructions

- Use the software template(s) and grading software provided in myCourses.

- Submit a single ZIP file to myCourses that must be named as your RIT user, e.g., `errvcs.zip`. Do not include '@rit.edu.' The file must contain folders named as the folders provided in the software template(s) containing your queries, Gradle or Poetry project(s).
- Everything will be graded on a Linux machine, so you must always use the exact names provided in this document, software template(s) and grading software.

Grading rubric

- All test cases in the grading software must pass with some restrictions like main memory and total running time.